

JAVA EMPLOYEE CONSTRUCTOR PROGRAM

Object-Oriented Programming Practical Record

1. Aim

To write a Java program using multiple constructors to initialize and display employee details.

2. Steps for Execution

1. Open VS Code, Eclipse or IntelliJ IDEA with JDK installed.
2. Create a Java file named EmployeeDemo.java.
3. Enter the Java source code and save the file.
4. Compile and run the program.
5. Create Employee objects using different constructors.
6. The default constructor initializes default employee details.
7. The second constructor initializes the employee name and ID.
8. The third constructor initializes all employee details.
9. Display the details of all employees.

3. Algorithm

1. Start the program.
2. Create the Employee class with name, id, designation, salary and promotionStatus.
3. Create a default constructor with default values.
4. Create a constructor with name and id.
5. Create a constructor with all employee details.
6. Create a display() method to print employee details.
7. Create three Employee objects using the three constructors.
8. Call display() for each object.
9. Stop the program.

4. Explanation of the Program

Employee class: It contains the employee data members name, id, designation, salary and promotionStatus.

Constructor 1: The default constructor assigns default values such as Unknown, 0, Not Assigned and 0.0.

Constructor 2: The constructor with name and id assigns the given name and ID and provides default designation, salary and promotion status.

Constructor 3: The constructor with all details assigns all the employee values supplied by the user.

display() method: It prints all the details of an employee.

Constructor Overloading: The Employee class has three constructors with different parameter lists. This demonstrates constructor overloading.

5. UML Class Diagram

```
+-----+
|           Employee           |
+-----+
| - name : String              |
| - id : int                   |
| - designation : String       |
| - salary : double            |
| - promotionStatus : String   |
+-----+
| + Employee()                 |
| + Employee(name, id)         |
| + Employee(name, id, designation, |
|           salary, promotionStatus) |
| + display() : void           |
+-----+
```

6. Java Source Code

```
class Employee {
    String name;
    int id;
    String designation;
    double salary;
    String promotionStatus;

    // Constructor 1: Default constructor
    Employee() {
        name = "Unknown";
        id = 0;
        designation = "Not Assigned";
        salary = 0.0;
        promotionStatus = "Not Eligible";
    }

    // Constructor 2: Constructor with name and id
    Employee(String name, int id) {
        this.name = name;
        this.id = id;
        designation = "Employee";
        salary = 20000;
        promotionStatus = "Pending";
    }

    // Constructor 3: Constructor with all employee details
    Employee(String name, int id, String designation,
        double salary, String promotionStatus) {
        this.name = name;
        this.id = id;
        this.designation = designation;
        this.salary = salary;
        this.promotionStatus = promotionStatus;
    }

    // Method to display employee details
    void display() {
        System.out.println("Name           : " + name);
        System.out.println("ID           : " + id);
        System.out.println("Designation      : " + designation);
        System.out.println("Salary          : " + salary);
        System.out.println("Promotion Status : " + promotionStatus);
        System.out.println("-----");
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {

        // Using default constructor
        Employee e1 = new Employee();

        // Using constructor with name and id
        Employee e2 = new Employee("Rahul", 101);

        // Using constructor with all details
        Employee e3 = new Employee(
            "Priya", 102, "Manager", 65000, "Eligible"
        );

        e1.display();
        e2.display();
        e3.display();
    }
}
```

7. Output of the Program

```
Name          : Unknown
ID            : 0
Designation   : Not Assigned
Salary        : 0.0
Promotion Status : Not Eligible
-----
```

```
Name          : Rahul
ID            : 101
Designation   : Employee
Salary        : 20000.0
Promotion Status : Pending
-----
```

```
Name          : Priya
ID            : 102
Designation   : Manager
Salary        : 65000.0
Promotion Status : Eligible
-----
```

8. Result

Thus, the Java program successfully demonstrates constructor overloading by creating Employee objects using three different constructors and displaying their details.